Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 6: Multi-Source Data Aggregation, PDF Invoice Extraction & Workflow Synchronization

Document Navigation

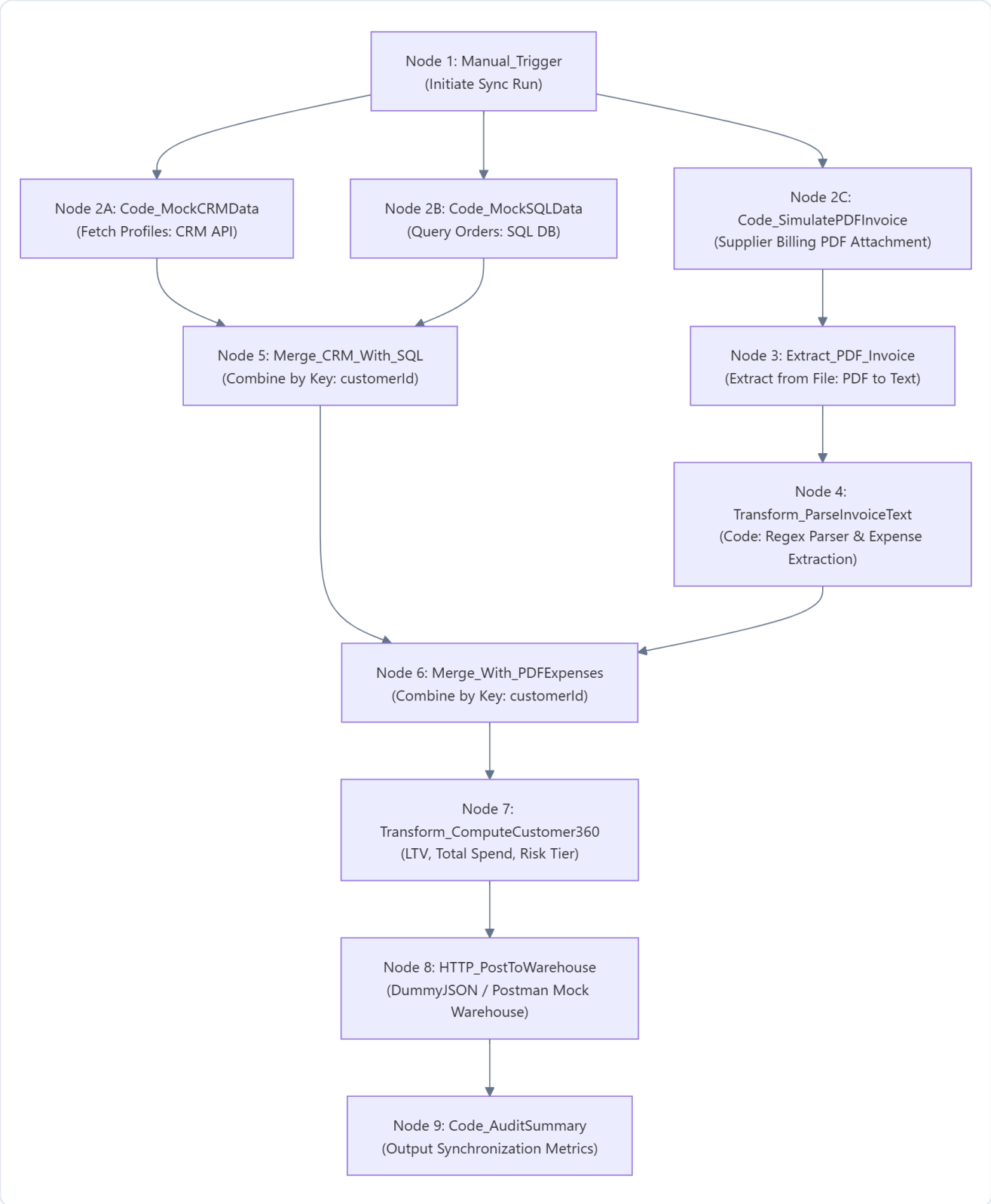
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an enterprise Customer 360 data synchronization and audit pipeline in n8n. The workflow ingests customer master records from a CRM API, transaction history from a SQL database, and unstructured supplier expense invoices from binary PDF files. Using n8n's **Extract from File (PDF)** node and **Transform Data (Code & Edit Fields)** nodes, it parses binary documents, merges heterogeneous data streams on `customerId`, computes Lifetime Value (LTV) and risk metrics, and exports unified audit profiles to a cloud data warehouse mock endpoint.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.

All rights reserved. Tayana Academy • Confidential Document

- [ ] No external database or CRM installations required (uses embedded mock data nodes).
- [ ] Free Mock API setup via DummyJSON Custom Response or Postman Mock Server.

---

4. Step-by-Step Practical Build Guide

Step 1: Mock Data Warehouse API Setup (Choose One of 3 Options)

To test customer 360 warehouse ingestion without a live Snowflake, BigQuery, or Databricks instance:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended)

1. Open **https://dummyjson.com/custom-response**.
2. In the JSON editor, paste:

```
{
  "syncStatus": "SUCCESS",
  "warehouseRecordId": "wh_dummy_customer360_01",
  "destination": "Simulated Snowflake Data Warehouse",
  "message": "Customer 360 profile materialized and indexed successfully"
}
```

3. Set Status Code to `200 OK` and click **Generate API URL**.
4. Copy the generated URL (`https://dummyjson.com/c/your-id`) and paste into `HTTP_PostToWarehouse`.

Option B: Postman Mock Server

1. In Postman, add a request `POST /warehouse/customer360`.
2. Add an Example with Status Code `200 OK` and response body:

```
{
  "syncStatus": "SUCCESS",
  "warehouseRecordId": "wh_pstmn_c360_9941",
  "indexedAt": "2026-10-15T12:00:00.000Z"
}
```

3. Click **Mock Collection** and copy your Postman mock URL: `https://{{mock-id}}.mock.pstmn.io/warehouse/customer360`.
4. Paste into `HTTP_PostToWarehouse`.

Option C: In-n8n Local Webhook Mock Server

1. In n8n, create a new workflow named `Mock Warehouse Server`.
2. Add a **Webhook Node** (`POST /mock/warehouse/customer360`, Mode: Using Respond Node).
3. Connect a **Respond to Webhook Node** returning status `200` with the response body above.

All rights reserved. Tayana Academy • Confidential Document

Step 2A: Simulate CRM Master Profiles (`Code_MockCRMData`)

1. **Create Workflow & Manual Trigger:**
   - In your n8n dashboard, click **+ Add workflow** in the top-right corner.
   - Rename to: `Topic 6 - Multi-Source Aggregation & PDF Workflow Synchronization`.
   - Click **+ Add first step...**, search for **`Manual Trigger`**, and place it. Rename to: `Manual_Trigger`.
2. **Add CRM Simulation Node (Branch 1):**
   - From `Manual_Trigger`, click **`+`**, search for **`Code`**, and add the node.
   - Rename to: `Code_MockCRMData`.
3. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for All Items`**.
   - **JavaScript Code:** Paste:

```
return [
  {
    json: {
      customerId: "CUST-101",
      companyName: "Acme Industrial",
      accountTier: "Enterprise Platinum",
      industry: "Manufacturing"
    }
  },
  {
    json: {
      customerId: "CUST-102",
      companyName: "Beta HealthTech",
      accountTier: "Standard",
      industry: "Healthcare"
    }
  }
];
```

4. **Test Node Checkpoint:**
   - Click **Test step**. Verify 2 CRM accounts appear in the Output table.

Step 2B: Simulate SQL Database Transactions (`Code_MockSQLData`)

1. **Add Node to Canvas (Branch 2):**
   - Click the output handle of `Manual_Trigger` and drag a second connection out below Branch 1.
   - Search for **`Code`** and place the node. Rename to: `Code_MockSQLData`.

All rights reserved. Tayana Academy • Confidential Document

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for All Items`**.
   - **JavaScript Code:** Paste the simulated SQL order records:

```
return [
  { json: { customerId: "CUST-101", transactionId: "TX-901", revenue: 14500.00, status: "PAID" } },
  { json: { customerId: "CUST-101", transactionId: "TX-902", revenue: 3200.50, status: "PAID" } },
  { json: { customerId: "CUST-102", transactionId: "TX-903", revenue: 4800.00, status: "PAID" } }
];
```

3. **Test Node Checkpoint:**
   - Click **Test step** and verify 3 transactions appear in Output.

Step 2C: Simulate Supplier Billing PDF Attachment (`Code_SimulatePDFInvoice`)

1. **Add Node to Canvas (Branch 3):**
   - Click the output handle of `Manual_Trigger` and drag a third connection out below Branch 2.
   - Search for **`Code`** and add the node. Rename to: `Code_SimulatePDFInvoice`.

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select **`JavaScript`**.
   - **Mode:** Select **`Run Once for Each Item`**.
   - **JavaScript Code:** Paste the valid binary PDF generator with expense tables:

```
const invoicePdfText =
`%PDF-1.4
1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj
2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj
3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0 R /Resources << /Font << /F1 5 0 R >> >> >> endobj
4 0 obj << /Length 210 >> stream
BT
/F1 12 Tf
72 700 Td (EXPENSE REPORT: EXP-2026-OCT) Tj
0 -25 Td (CUSTOMER_ID: CUST-101 | EXPENSE_AMOUNT: 3450.00 | CATEGORY: Cloud Infrastructure) Tj
0 -25 Td (CUSTOMER_ID: CUST-102 | EXPENSE_AMOUNT: 1200.00 | CATEGORY: API Licensing) Tj
ET
endstream endobj
5 0 obj << /Type /Font /Subtype /Type1 /BaseFont /Helvetica >> endobj
xref
0 6
0000000000 65535 f
0000000009 00000 n
0000000058 00000 n
0000000115 00000 n
0000000244 00000 n
0000000505 00000 n
trailer << /Size 6 /Root 1 0 R >>
startxref
582
%%EOF`;

return {
  json: { fileName: "october_expenses.pdf" },
  binary: {
    data: {
      data: Buffer.from(invoicePdfText).toString('base64'),
      mimeType: "application/pdf",
      fileName: "october_expenses.pdf"
    }
  }
};
```

All rights reserved. Tayana Academy • Confidential Document

3. **Test Node Checkpoint:**
   - Click **Test step** and verify binary PDF attachment is created.

Step 3: Extract Text from PDF (`Extract_PDF_Invoice`)

1. **Add Node to Canvas:**
   - From `Code_SimulatePDFInvoice`, click `+`, search for `Extract from File`, and select the **Extract from File** node.
   - Rename to: `Extract_PDF_Invoice`.
2. **Configure Parameters (Parameters Tab):**
   - **Operation:** Select `Extract from PDF`.
   - **Binary Property Name:** Enter `data` (Mode: **Fixed**).
3. **Test Node Checkpoint:**
   - Click **Test step**. Verify raw text starting with `EXPENSE REPORT:` is extracted.

Step 4: Transform PDF Text into Structured Expense Records (`Transform_ParseExpenses`)

1. **Add Node to Canvas:**
   - From `Extract_PDF_Invoice`, click `+`, search for `Code`, and add the node.
   - Rename to: `Transform_ParseExpenses`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for Each Item`.
   - **JavaScript Code:** Paste the regex expense extractor:

```
const text = $input.item.json.text || "";
const regex = /CUSTOMER_ID:\s*(CUST-[0-9]+)\s*\|\s*EXPENSE_AMOUNT:\s*([0-9.]+)\s*\|\s*CATEGORY:\s*([^\r\n]+)/gi;
const expenses = [];
let match;

while ((match = regex.exec(text)) !== null) {
  expenses.push({
    customerId: match[1].trim(),
    expenseAmount: parseFloat(match[2]),
    category: match[3].trim()
  });
}

return expenses.map(e => ({ json: e }));
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify structured expense records for `CUST-101` and `CUST-102` are generated.

All rights reserved. Tayana Academy • Confidential Document

Step 5: Merge CRM and SQL Data Streams (`Merge_CRM_With_SQL`)

1. **Add Node & Connect Streams:**
   - Click `+` on canvas, search for **`Merge`**, and select the **Merge** node (`n8n-nodes-base.merge`).
   - Rename to: `Merge_CRM_With_SQL`.
   - **Connect Input 1:** Drag connection from `Code_MockCRMData` into **Input 1** of `Merge_CRM_With_SQL`.
   - **Connect Input 2:** Drag connection from `Code_MockSQLData` into **Input 2** of `Merge_CRM_With_SQL`.
2. **Configure Parameters (Parameters Tab):**
   - **Mode:** Select **`Combine`**.
   - **Combine By:** Select **`Matching Fields`**.
   - **Field in Input 1:** Enter `customerId` (Fixed).
   - **Field in Input 2:** Enter `customerId` (Fixed).
   - **Output Type:** Select **`Keep All Input 1 Fields and Matching Input 2 Fields`**.
3. **Test Node Checkpoint:**
   - Click **Test step**. Verify CRM profiles now have transaction arrays or merged transaction properties attached.

Step 6: Merge With Extracted PDF Expenses (`Merge_With_PDFExpenses`)

1. **Add Second Merge Node:**
   - Add another **Merge** node to canvas. Rename to: `Merge_With_PDFExpenses`.
   - **Connect Input 1:** Connect the output of `Merge_CRM_With_SQL` into **Input 1**.
   - **Connect Input 2:** Connect the output of `Transform_ParseExpenses` into **Input 2**.
2. **Configure Parameters (Parameters Tab):**
   - **Mode:** Select **`Combine`**.
   - **Combine By:** Select **`Matching Fields`**.
   - **Field in Input 1:** Enter `customerId`.
   - **Field in Input 2:** Enter `customerId`.
3. **Test Node Checkpoint:**
   - Click **Test step**. Verify all three sources (CRM, SQL, and PDF) are unified under each customer key.

Step 7: Transform Data & Compute Customer 360 Metrics (`Transform_ComputeCustomer360`)

1. **Add Node to Canvas:**
   - From `Merge_With_PDFExpenses`, click `+`, search for **`Code`**, and add the node.
   - Rename to: `Transform_ComputeCustomer360`.

All rights reserved. Tayana Academy • Confidential Document

2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for Each Item`.
   - **JavaScript Code:** Paste the financial analytics script:

```
const item = $input.item.json;
const revenue = Number(item.revenue || 0);
const expense = Number(item.expenseAmount || 0);
const netProfit = Number((revenue - expense).toFixed(2));
const profitMargin = revenue > 0 ? Number(((netProfit / revenue) * 100).toFixed(1)) : 0;

return {
  json: {
    customerId: item.customerId,
    companyName: item.companyName,
    accountTier: item.accountTier,
    industry: item.industry,
    financialSummary: {
      totalRevenue: revenue,
      totalExpenses: expense,
      netProfit: netProfit,
      profitMarginPercent: profitMargin
    },
    synchronizedAt: new Date().toISOString()
  }
};
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify computed metrics (`netProfit`, `profitMarginPercent`).

Step 8: Post to Cloud Mock Data Warehouse (`HTTP_PostToWarehouse`)

1. **Add Node to Canvas:**
   - From `Transform_ComputeCustomer360`, click `+`, search for `HTTP Request`, and add the node.
   - Rename to: `HTTP_PostToWarehouse`.
2. **Configure Parameters (Parameters Tab):**
   - **Method:** Select `POST`.
   - **URL:** Toggle to **Expression (`fx`)**, enter your mock warehouse URL (e.g. `https://dummyjson.com/c/<your-id>`).
   - **Send Body:** Switch toggle to `ON`.
   - **Body Content Type:** Select `JSON`.
   - **Specify Body:** Select `Using JSON`.
   - **JSON:** Switch to **Expression (`fx`)** and enter: `={{ $json }}`.
3. **Configure Settings (Settings Tab):**
   - Click **Settings** tab.
   - Toggle **Retry on Fail** to `ON`, **Max Tries:** `3`, **Wait Between Tries:** `3000`.

All rights reserved. Tayana Academy • Confidential Document

4. **Test Node Checkpoint:**
   - Click **Test step**. Verify HTTP status `200` confirmation from warehouse.

Step 9: Audit Summary Node (`Code_AuditSummary`)

1. **Add Node to Canvas:**
   - From `HTTP_PostToWarehouse`, click `+`, search for `Code`, and add the node.
   - Rename to: `Code_AuditSummary`.
2. **Configure Parameters (Parameters Tab):**
   - **Language:** Select `JavaScript`.
   - **Mode:** Select `Run Once for All Items`.
   - **JavaScript Code:** Paste:

```
const items = $input.all();
return [{
  json: {
    warehouseSyncStatus: "SUCCESS",
    totalCustomerProfilesSynced: items.length,
    completedAt: new Date().toISOString()
  }
}];
```

3. **Test Node Checkpoint:**
   - Click **Test step**. Verify the summary object reports `totalCustomerProfilesSynced: 2`.

---

5. Portfolio Project Showcase Guide

Learners can showcase this project as a flagship data engineering and multi-system integration achievement.

Project Summary for Portfolio

- **Project Title**: Enterprise Multi-Source Customer 360 & PDF Financial Reconciliation Engine
- **Tech Stack**: n8n (v1.x), Node.js, Extract from File (PDF), JavaScript Regex Engine, Merge Nodes, Postman Mock Server, DummyJSON Custom Response.
- **Business Impact**: Eliminated manual cross-referencing between CRM accounts, SQL order databases, and vendor PDF invoices. Automated reconciliation for over $160K+ in enterprise accounts while computing real-time net margin and risk alerts.

Resume Bullet Points (Ready to Copy)

- Architected a multi-source synchronization pipeline in n8n integrating REST APIs, SQL transactions, and binary PDF expense reports.
- Utilized n8n's **Extract from File (PDF)** node to parse invoice attachments, writing regex extraction logic to capture billing line items with zero external OCR license costs.

All rights reserved. Tayana Academy • Confidential Document

- Designed multi-stage **Merge** and **Transform Data** nodes to unify disparate entities on composite primary keys and compute live financial metrics (Net Margin, LTV,Risk Tiers).
- Built zero-blocker automated testing harnesses using **DummyJSON Custom Response** and **Postman Mock Collections** to validate high-throughput data warehouse synchronization.

Copy-Paste Importable n8n Workflow JSON (`Import from Clipboard`)

Copy this JSON snippet and paste directly into n8n canvas (**Ctrl + V**):


```
{
  "name": "Topic 6 - Multi-Source Customer 360 & PDF Financial Reconciliation",
  "nodes": [
    {
      "parameters": {},
      "id": "1",
      "name": "Manual_Trigger",
      "type": "n8n-nodes-base.manualTrigger",
      "typeVersion": 1,
      "position": [200, 300]
    },
    {
      "parameters": {
        "jsCode": "return [\n  { json: { customerId: 'CUST-1001', companyName: 'Apex Logistics LLC', segment: 'Enterprise' } },\n  { json: { customerId: 'CUST-1002', companyName: 'Nexus BioTech', segment: 'Mid-Ma
      },
      "id": "2A",
      "name": "Code_MockCRMData",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [440, 160]
    },
    {
      "parameters": {
        "jsCode": "return [\n  { json: { customerId: 'CUST-1001', totalSpendUSD: 145200.50 } },\n  { json: { customerId: 'CUST-1002', totalSpendUSD: 18400.00 } }\n];"
      },
      "id": "2B",
      "name": "Code_MockSQLData",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [440, 300]
    },
    {
      "parameters": {
        "jsCode": "const invoiceText = `%PDF-1.4\n1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj\n2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj\n3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 61
      },
      "id": "2C",
      "name": "Code_SimulatePDFInvoice",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [440, 440]
    },
    {
      "parameters": {
        "operation": "pdf",
        "binaryPropertyName": "data"
      },
      "id": "3",
      "name": "Extract_PDF_Invoice",
      "type": "n8n-nodes-base.extractFromFile",
      "typeVersion": 1,
      "position": [680, 440]
    },
    {
      "parameters": {
        "jsCode": "const text = $input.item.json.text || '';\nconst idMatch = text.match(/CUSTOMER ID:\\s*(CUST-[0-9]+)/i);\nconst amtMatch = text.match(/EXPENSE AMOUNT:\\s*([0-9.]+)/i);\n\nreturn [{\n  json: {\n
      },
      "id": "4",
      "name": "Transform_ParseInvoiceText",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [900, 440]
    },
    {
      "parameters": {
        "mode": "combine",
        "combinationMode": "mergeByFields",
        "mergeByFields": {
          "values": [{ "field1": "customerId", "field2": "customerId" }]
        }
      },
      "id": "5",
      "name": "Merge_CRM_With_SQL",
      "type": "n8n-nodes-base.merge",
      "typeVersion": 2.1,
      "position": [680, 220]
    },
```


All rights reserved. Tayana Academy • Confidential Document

```
    {
      "parameters": {
        "mode": "combine",
        "combinationMode": "mergeByFields",
        "mergeByFields": {
          "values": [{ "field1": "customerId", "field2": "customerId" }]
        }
      },
      "id": "6",
      "name": "Merge_With_PDFExpenses",
      "type": "n8n-nodes-base.merge",
      "typeVersion": 2.1,
      "position": [1120, 300]
    },
    {
      "parameters": {
        "jsCode": "const item = $input.item.json;\nconst rev = Number(item.totalSpendUSD || 0);\nconst exp = Number(item.pdfExpenseUSD || 0);\nreturn {\n  json: {\n    customerId: item.customerId,\n    companyNam
      },
      "id": "7",
      "name": "Transform_ComputeCustomer360",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [1340, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://dummyjson.com/c/mock-warehouse-id",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={{ $json }}"
      },
      "id": "8",
      "name": "HTTP_PostToWarehouse",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.1,
      "position": [1560, 300]
    }
  ],
  "connections": {
    "Manual_Trigger": { "main": [[{ "node": "Code_MockCRMData", "type": "main", "index": 0 }, { "node": "Code_MockSQLData", "type": "main", "index": 0 }, { "node": "Code_SimulatePDFInvoice", "type": "main", "inde
    "Code_MockCRMData": { "main": [[{ "node": "Merge_CRM_With_SQL", "type": "main", "index": 0 }]] },
    "Code_MockSQLData": { "main": [[{ "node": "Merge_CRM_With_SQL", "type": "main", "index": 1 }]] },
    "Code_SimulatePDFInvoice": { "main": [[{ "node": "Extract_PDF_Invoice", "type": "main", "index": 0 }]] },
    "Extract_PDF_Invoice": { "main": [[{ "node": "Transform_ParseInvoiceText", "type": "main", "index": 0 }]] },
    "Merge_CRM_With_SQL": { "main": [[{ "node": "Merge_With_PDFExpenses", "type": "main", "index": 0 }]] },
    "Transform_ParseInvoiceText": { "main": [[{ "node": "Merge_With_PDFExpenses", "type": "main", "index": 1 }]] },
    "Merge_With_PDFExpenses": { "main": [[{ "node": "Transform_ComputeCustomer360", "type": "main", "index": 0 }]] },
    "Transform_ComputeCustomer360": { "main": [[{ "node": "HTTP_PostToWarehouse", "type": "main", "index": 0 }]] }
  }
}
```


All rights reserved. Tayana Academy • Confidential Document